CAMINO A LA LIGA

Alumno/a	Cristian Magro Rodríguez
Grupo	1ºDAW
Curso	2026-2027
Asignatura	Programación
Repositorio GitHub	https://github.com/cristianmagrorodriguez/CaminoLiga

1. Introducción y descripción general

CAMINO A LA LIGA es un juego de consola programado en Java. El usuario comienza eligiendo uno de ocho clubes de fútbol. Después puede consultar su plantilla, seleccionar once jugadores, disputar partidos simulados y consultar una clasificación con los partidos jugados y los puntos obtenidos por cada equipo.

La interacción se realiza mediante teclado y pantalla. Para leer las opciones se utiliza la clase `Scanner`. La ejecución se organiza mediante un menú que se repite hasta que el usuario selecciona la opción de salir.

El proyecto permite practicar conceptos básicos de programación: variables, arrays, matrices, bucles, condicionales, selección múltiple con `switch`, generación de números aleatorios y validación de entradas.

1.1. Objetivo del juego

El objetivo es dirigir uno de los equipos disponibles. El jugador escoge un club, forma un once titular y disputa encuentros contra rivales seleccionados aleatoriamente. Cada resultado modifica la puntuación del equipo elegido: una victoria suma tres puntos, un empate suma uno y una derrota no suma puntos.

1.2. Equipos disponibles

El programa incluye Real Madrid, Barcelona, Liverpool, Manchester City, PSG, Roma, Juventus y Atlético de Madrid. Cada club tiene asociada una plantilla de veinte jugadores. Los nombres están escritos directamente en el código fuente.

1.3. Tipo de aplicación

Es una aplicación de consola, no gráfica. No utiliza base de datos ni guarda resultados en ficheros. Por tanto, al cerrar el programa se pierden los datos acumulados durante la sesión.

2. Reglas y desarrollo de una partida

2.1. Inicio y elección del club

Al iniciar aparece el título CAMINO A LA LIGA y un mensaje de bienvenida. El usuario pulsa ENTER para continuar. Se muestra una lista numerada del 1 al 8.

2.2. Menú principal

Una vez elegido el equipo, aparece un menú con cinco opciones:

Opción	Acción	Descripción
1	Ver plantilla	Muestra los veinte jugadores del club.
2	Elegir 11 titular	Permite seleccionar once jugadores distintos.
3	Jugar partido	Genera un rival y un marcador aleatorios.
4	Clasificación	Muestra partidos jugados y puntos obtenidos.
5	Salir	Finaliza el juego

2.3. Selección de titulares

El usuario elige once números de la plantilla. El programa comprueba que cada número esté entre 1 y 20 y que el jugador no haya sido seleccionado previamente. Los nombres elegidos se guardan en `titulares` y, al terminar, se muestran por pantalla.

2.4. Partido y resultado

Para jugar es obligatorio haber seleccionado previamente el once titular. El rival se obtiene al azar entre los ocho equipos, repitiendo el sorteo si coincide con el equipo del usuario. También se generan dos cantidades aleatorias de goles, entre 0 y 4, se compara los goles de cada equipo y así se obtiene el resultado del partido.

2.5. Puntuación y finalización

Tras mostrar el marcador, aumenta el contador de partidos del equipo del usuario. Si gana suma tres puntos, si empata suma uno y si pierde no suma. No existe una condición automática de campeón, descenso o derrota definitiva: el usuario puede seguir jugando hasta elegir «Salir».

3. Funcionalidades implementadas

Selección de equipo. Permite escoger uno de los ocho clubes y conserva su índice durante el resto de la ejecución.

Consulta de plantilla. Recorre el array de jugadores correspondiente al equipo seleccionado y muestra sus veinte integrantes.

Creación del once titular. Solicita once jugadores y evita repetir una misma posición de la plantilla durante esa selección.

Simulación de partidos. Elige un rival distinto al usuario y genera un marcador aleatorio.

Sistema de puntos. Aplica tres puntos por victoria, uno por empate y cero por derrota.

Clasificación. Recorre los arrays de equipos, partidos y puntos para presentar los datos almacenados.(Solo se suma en el equipo del usuario).

Menú repetitivo. Permite realizar varias acciones y volver al menú después de cada una. La opción 5 termina el juego.

3.1. Diferencias respecto a la propuesta inicial

En la versión actual, el juego funciona mediante consola, usa plantillas escritas en el código y genera los partidos de forma aleatoria.

Diferencias, tenía pensado que los jugadores tuviesen atributos de ATQ,DF y que eso también influyera en el resultado, pero no he sabido como hacerlo.

3.2. Alcance actual

La partida permite acumular resultados del club elegido durante la ejecución. Aunque la clasificación muestra los ocho equipos, los arrays de puntos y partidos solo se modifican en la posición del equipo controlado por el usuario.
Los demás clubes permanecen con cero partidos y cero puntos.

4. Diseño: variables y constantes

El programa se desarrolla dentro de la clase `CaminoALaLiga` y toda la lógica se encuentra en el método `main`. No se declaran constantes,solo variables.

4.1. Variables principales

Variable	Tipo	Uso
sc	Scanner	Lee los datos introducidos por teclado.
equipo	int	Índice del equipo elegido, entre 0 y 7.
opcion	int	Opción del menú principal.
puntos	int[]	Puntos asociados a cada equipo.
partidos	int[]	Partidos disputados por cada equipo.
titulares	String[]	Guarda los once nombres seleccionados.
elegidos	boolean[]	Marca las posiciones de jugadores ya seleccionadas.
rival	int	Índice del equipo rival aleatorio.
goles1 / goles2	int	Goles del usuario y del rival.

4.2. Arrays frente a constantes

Los arrays `puntos` y `partidos` se crean con una longitud igual a `equipos.length`. Así cada posición representa al mismo club en los tres arrays. También se utiliza `equipos.length` para calcular el rango de selección aleatoria del rival.

4.3. Tipos de datos

Se utilizan enteros para índices, opciones, goles, partidos y puntos,cadenas de texto para los nombres de clubes y jugadores y valores booleanos para indicar si un jugador ya está elegido. Esta combinación permite representar información textual y estados de selección.

5. Estructuras de control

5.1. Bucles for

Los bucles `for` recorren arrays y repiten acciones un número conocido de veces. Se utilizan para mostrar los ocho equipos, listar la plantilla, pedir los once titulares, mostrar el once seleccionado y recorrer la clasificación.

5.2. Bucles while

Se usan para validar la elección del equipo y para impedir que el rival sea el mismo club que el usuario. También se utiliza un `while` en la selección de titulares para rechazar números fuera del rango o jugadores repetidos. El bucle vuelve a pedir un dato hasta que se cumpla la condición válida.

5.3. Bucle do-while

El menú principal está dentro de un `do-while`. Esto garantiza que se muestre al menos una vez.

5.4. Condicionales if / else if / else

Los condicionales comprueban si el equipo ha ganado, empatado o perdido. También se utilizan para comprobar si se han elegido los titulares antes de permitir jugar un partido.

5.5. Estructura switch

El `switch` organiza las cinco opciones del menú. Cada `case` ejecuta una acción diferente y termina con `break` para evitar que se ejecuten los casos siguientes.

5.6. Generación aleatoria

La clase `Math.random` genera el rival y los goles. La conversión a `int` elimina la parte decimal y permite obtener índices y marcadores enteros.

6. Explicación detallada de los arrays

Los arrays son fundamentales porque permiten almacenar listas de datos y relacionar información mediante posiciones. En Java, el primer elemento ocupa la posición 0.

6.1. Array equipos

Es un array unidimensional de tipo `String` con ocho posiciones. Cada posición contiene el nombre de un club. Por ejemplo, `equipos[0]` contiene «Real Madrid» y `equipos[7]` contiene «Atletico de Madrid». El número que introduce el usuario se transforma en índice restando uno.

6.2. Matriz jugadores

Es un array bidimensional de cadenas: `String[][]`. La primera dimensión identifica el equipo y la segunda identifica al jugador dentro de la plantilla. Se puede interpretar como una tabla de ocho filas y veinte columnas. Por ejemplo, `jugadores[0][0]` corresponde al primer jugador del Real Madrid.

Cuando el usuario elige un equipo, `jugadores[equipo]` selecciona la fila completa de ese club. Después, `jugadores[equipo][i]` accede al jugador situado en la columna `i`. Esta estructura evita crear ocho arrays independientes.

6.3. Arrays puntos y partidos

Son dos arrays unidimensionales de enteros, ambos con ocho posiciones. Se relacionan con `equipos` por el mismo índice: `puntos[0]` y `partidos[0]` almacenan los datos del Real Madrid, que ocupa la posición 0. Al comenzar, los enteros se inicializan automáticamente a cero.

6.4. Array titulares

Es un array de once cadenas. Guarda los nombres seleccionados. Al principio sus posiciones contienen `null`, porque todavía no se ha asignado ningún nombre. El programa comprueba `titulares[0] == null` para detectar que aún no se ha realizado una primera selección.

6.5. Array elegidos

Es un array de veinte valores booleanos, creado con `new boolean[jugadores[0].length]`. Cada posición corresponde al jugador de la misma posición en la plantilla. Si se elige el jugador número 4, se marca `elegidos[3] = true`. Así el programa sabe que no debe aceptar otra vez ese jugador.

El array booleano no guarda nombres, sino su estado: `true` (ya elegido) y `false` (no elegido).

7. Relación entre arrays y selección del once

La lógica depende de que varios arrays compartan el mismo sistema de índices. La posición de un equipo en `equipos` se utiliza también para acceder a su puntuación y a su número de partidos. Esta relación se conoce como correspondencia por índice.

Por ejemplo, si el usuario elige la opción 2 al seleccionar club, el programa guarda el índice 1 en `equipo`. A partir de ese momento consulta `equipos[1]`, muestra `jugadores[1]` y actualiza `puntos[1]` y `partidos[1]` cuando juega.

7.1. Selección de once jugadores

El bucle de selección se repite once veces. En cada vuelta se pide un número del 1 al 20. El programa resta uno para convertirlo en índice. La condición de validación comprueba que el número no sea menor que 1 y que no sea mayor que 20 y que la posición correspondiente no esté marcada como elegida.

Una vez aceptado el número, se marca la posición del array booleano y se copia el nombre a la siguiente posición libre del array de titulares. De esta forma, el orden de selección también queda almacenado.

7.2. Correspondencia de índices

Índice	Equipo	Array de jugadores	Puntos / partidos
0	Real Madrid	jugadores[0]	puntos[0] / partidos[0]
1	Barcelona	jugadores[1]	puntos[1] / partidos[1]
…	…	…	…
7	Atletico de Madrid	jugadores[7]	puntos[7] / partidos[7]

8. Pruebas del programa

8.1. Casos normales

Prueba	Entrada / acción	Resultado esperado
Elegir equipo	Introducir 1	Selecciona Real Madrid y muestra el menú.
Ver plantilla	Opción 1	Muestra los 20 jugadores.
Once titular	11 números distintos del 1 al 20	Guarda y muestra los once nombres.
Jugar partido	Opción 3 tras crear el once	Muestra rival distinto y marcador.
Victoria	Más goles equipo usuario	Suma 3 puntos.
Empate	Mismos goles	Suma 1 punto.
Derrota	Menos goles equipo usuario	No suma puntos.
Salir	Opción 5	Termina el juego.

8.2. Casos incorrectos y límite

Prueba	Entrada / acción	Comportamiento según el código
Equipo fuera de rango	0 o 9	Vuelve a pedir un número entre 1 y 8.
Opción incorrecta	Número distinto de 1 a 5	Muestra «Opcion incorrecta» y vuelve al menú.
Jugador fuera de rango	0 o 21 o negativos	Vuelve a pedir un número entre 1 y 20.
Jugador repetido	Repetir un número	Opción incorrecta
Jugar sin titulares	Opción 3 al inicio	Indica que primero hay que elegir once.

9. Dificultades encontradas y soluciones

9.1. Entrada de texto cuando se espera un número

Durante las pruebas con otra persona se detectó que, al pedir el número de un jugador, introducir un valor como «10a» provoca un error.

La solución propuesta es comprobar primero el tipo de entrada mediante `hasNextInt()`.

9.2. Clasificación incompleta

La clasificación muestra todos los clubes, pero el programa solo actualiza los datos del equipo seleccionado por el usuario. No representa todavía una liga completa con resultados de todos los equipos. Una mejora sería simular también los demás encuentros y actualizar los arrays de ambos participantes.

9.3. Robustez

Además de validar el rango, conviene controlar las entradas no numéricas, los cambios de alineación y los intentos de jugar sin titulares. La validación debe evitar que una entrada incorrecta deje el programa en un estado inesperado.

10. Posibles ampliaciones para la segunda evaluación

El proyecto puede ampliarse para que la experiencia se parezca más a una temporada de liga y para mejorar la calidad del código.

1. **Validación completa de entradas.** Aplicar `hasNextInt()`.

2. **Permitir cambiar el once.** Reiniciar el array `elegidos` antes de crear una nueva alineación y permitir consultar la alineación actual.

3. **Liga completa.** Simular partidos entre todos los equipos, actualizar los puntos de ambos clubes y calcular una clasificación real.

4. **Calendario de jornadas.** Organizar los encuentros por jornadas, evitando que un equipo juegue dos veces en la misma jornada y permitiendo consultar los partidos pendientes.

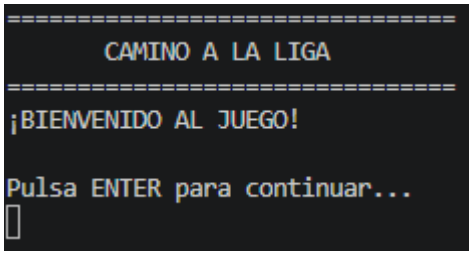
5. **Condición de campeón.** Establecer un número de jornadas y declarar campeón al equipo con más puntos al finalizar la temporada.

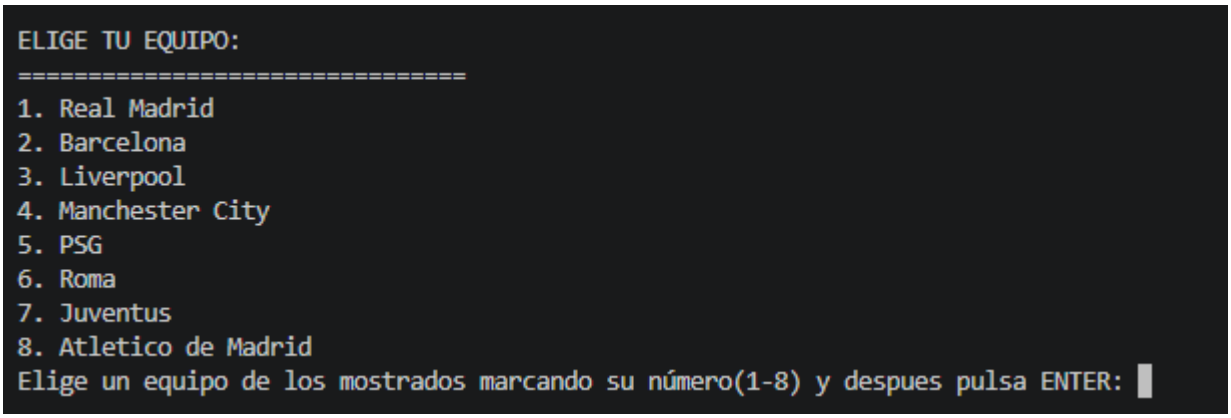
6. **Guardado de partidas.** Guardar los resultados en un archivo para continuar la temporada después de cerrar el programa.

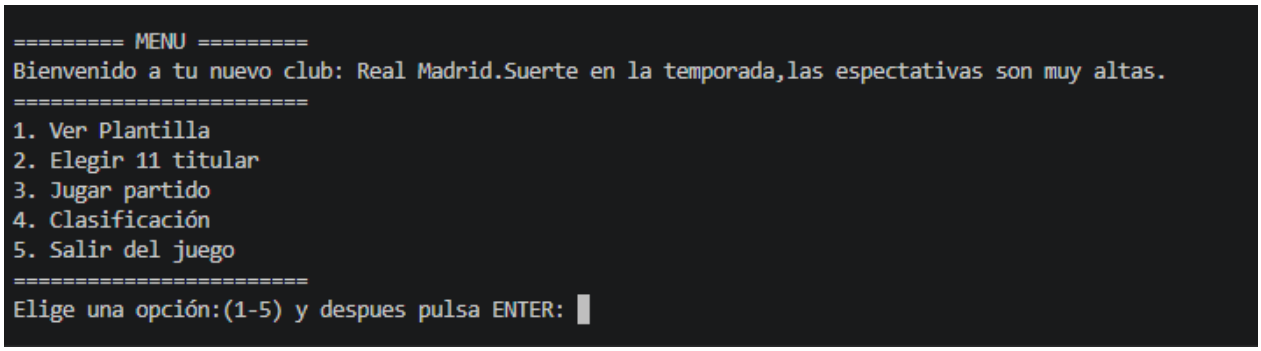
10.1. Conclusión

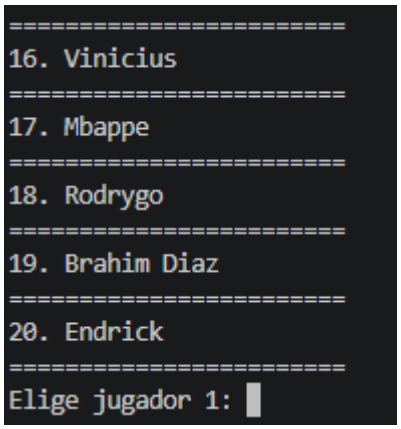
CAMINO A LA LIGA permite practicar estructuras fundamentales de Java mediante un juego de fútbol sencillo e interactivo. El uso de arrays facilita almacenar equipos, plantillas, titulares y estadísticas. Los bucles y condicionales controlan la navegación y las reglas de los partidos. Durante la revisión se han identificado mejoras relacionadas con la validación de datos, la gestión de alineaciones y el cálculo de una clasificación completa. Estas mejoras ofrecen una base concreta para continuar el proyecto en la segunda evaluación.

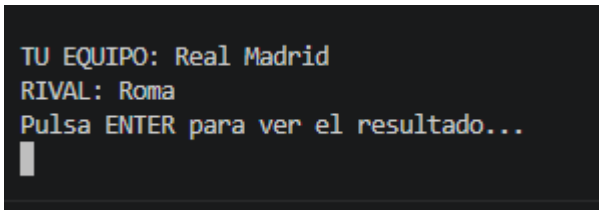
11.Capturas de pantalla

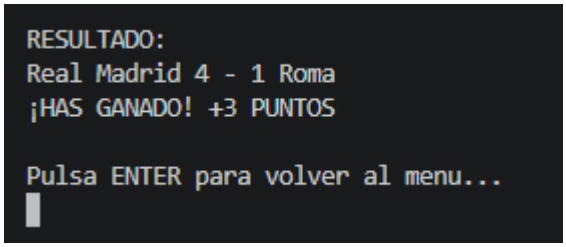







==========
16. Vinicius
==========
17. Mbappe
==========
18. Rodrygo
==========
19. Brahim Diaz
==========
20. Endrick
==========
Elige jugador 1:


TU EQUIPO: Real Madrid
RIVAL: Roma
Pulsa ENTER para ver el resultado...


RESULTADO:
Real Madrid 4 - 1 Roma
¡HAS GANADO! +3 PUNTOS
Pulsa ENTER para volver al menu...